

# 面向教育播客生成的多阶段 Agent workflows 设计与原型实现

选题编号: E (工程实践与设计)

答 研 522025330141

南京大学计算机学院

**摘 要:** 教育类播客和视频制作需要将长篇知识材料转化为听众能够理解的对谈内容, 通常依赖人工完成选题拆解、编导规划、脚本撰写和后期制作。直接使用大模型生成完整长脚本, 容易将知识选择、叙事结构、角色表达和媒体组织混在一次调用中, 导致内容结构不可控、事实约束不足且难以调试。本文围绕选题 E “工程实践与设计”, 整理并分析我在项目中实现的一个教育播客生成原型。该 workflow 将输入章节依次转化为主题规划、编导大纲、双人对谈脚本、配音字幕、画面计划以及视频产物。案例产物表明, 这种拆分方式能够保留中间结果, 支持阶段化检查和后续人工审核, 为知识型内容生产提供了一种可复用的 Agent 工程实践思路。

**关键词:** 大模型智能体; 多智能体协作; workflow 编排; 教育播客; 检索增强生成; 媒体生成

## 1 引言

知识型播客和视频不能只把参考材料朗读出来。以哲学史等抽象学科为例, 一期面向零基础听众的节目往往需要先选择合适的切入问题, 再把概念拆成由浅入深的解释链, 并通过提问、追问、类比和总结维持听众注意力。传统制作流程通常需要编导、内容专家、主持人和后期人员协作完成, 人工成本较高。

大模型为这类内容生产带来了新的可能, 但直接提示模型“写一整期播客脚本”并不稳定。长脚本生成同时包含资料理解、结构安排、角色扮演和语言润色等任务, 模型容易在长上下文中出现主题漂移, 或者在尚未明确节目结构时提前生成大量台词。ReAct 等研究指出, 语言模型可以通过交替生成推理轨迹和行动来增强任务执行过程的可解释性和可控性 [1]。这一思路启发我们将“长文本一次生成”改造为“多阶段、短任务、可检查”的 workflow。

本文关注点放在 Agent 系统的工程组织方式上。系统以教育播客为目标场景, 将一章参考资料依次处理为主题、编导计划、双人脚本和媒体产物接口。通过这种设计, 复杂内容生产被拆分为多个职责清晰的阶段, 每个阶段都具有明确输入、输出和质量检查点。

## 2 需求分析与设计目标

教育播客生成系统首先要服务听众理解, 自动化只是实现手段。对于零基础听众, 内容需要从直观问题出发, 再逐步引入抽象概念; 对于对谈形式, 脚本需要体现自然提问、解释、追问和转场; 对于工程系统, 生成过程还需要能够被检查和重跑, 避免所有质量问题都堆积到最终脚本阶段。

从制作流程看, 一期高质量知识播客至少包含三类角色。第一类是“知识锚点”, 负责保证概念准确和逻辑深度; 第二类是“听众嘴替”, 代表听众提出困惑和追问; 第三类是“幕后统筹”, 负责组织节目节奏、段落意图和转场方式。本文的系统设计将这些角色转化为 Agent workflow 中的不同阶段: Topic 承担选题和主题拆解, Director 承担编导规划, Agent Speechers 承担双主持人对谈生成, Voice、Image 和 Editor 则负责后续媒体表达。

据此, 系统设计目标包括四点。第一, 内容组织应符合从浅入深的教育逻辑; 第二, 对谈脚本应具有角色分工和自然衔接; 第三, 每个阶段应保留结构化中间产物, 便于人工检查; 第四, 媒体生成阶段应与内容阶段解耦, 使语音、画面和剪辑模块能够被替换或扩展。

### 3 系统总体架构

系统总体上分为知识层、内容层和表现层。知识层主要对应参考章节、外部资料和 RAG 检索。对于知识型内容生成，RAG 通过结合外部检索结果与参数化模型，有助于缓解事实漂移和知识遗漏问题 [2]。在本项目中，知识层负责提供资料依据：例如 Agent Speechers 阶段会根据主题的 zero-to-hero 逻辑并发查询参考资料，并把返回内容放入提示词的 context 区域，供后续对话生成引用。

内容层是系统的核心，包含 Topic、Director 和 Agent Speechers 三个阶段。Topic 将章节资料拆解为递进主题；Director 为每个主题生成编导计划，明确讨论目的、提示素材和转场方向；Agent Speechers 根据编导计划生成双主持人对话脚本。表现层包括 Voice、Image 和 Editor，主要解决声音、画面、字幕和剪辑呈现问题，使内容层的脚本进一步变成可观看的视频结果。

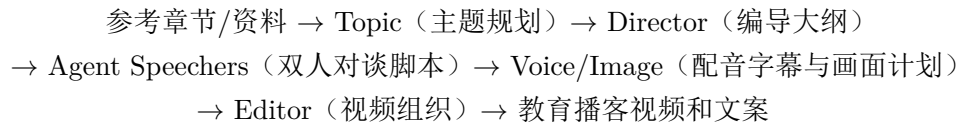


图 1: 系统总体 workflow

我采用这种架构的主要原因，是希望把“生成一档节目”拆为多个可观察的转换步骤。每个阶段都只处理相对聚焦的问题，并将结果交给下一阶段。相比一次性生成完整脚本，分阶段 workflow 更适合插入人工审核，也便于替换底层模型或外部服务。

### 4 多阶段 Agent workflow 设计

#### 4.1 Topic: 从章节到递进主题

Topic 阶段的输入是参考章节或其他知识材料，输出是一组具有教育递进关系的主题。每个主题不仅包含名称和核心概念，还应包含从零基础问题走向深层理解的逻辑，即 zero-to-hero 路径。这样做的目的，是将长文本生成中最困难的宏观结构问题前置处理。

在原型示例中，系统将参考内容拆为“从自然追问到人的转向”和“苏格拉底与可对话的智能体”等主题。前者从古希腊自然哲学对世界本原的追问出发，逐步过渡到人、知识与道德问题；后者则把苏格拉底式问答与可对话 Agent 的协作机制联系起来。这样的主题划分既服务课程内容，也服务报告中的 Agent 技术分析。

#### 4.2 Director: 从主题到编导计划

Director 阶段先不生成台词，转而把主题转化为可执行的编导大纲。它关注每段讨论的目的、可用素材和转场提示，相当于把传统制作流程中的“幕后统筹”显式化。这样做可以降低脚本生成阶段的自由度，使对谈 Agent 不必同时承担宏观结构规划和具体语言表达。

原型中，Director 为“从自然追问到人的转向”生成“问题导入”阶段，并给出“用世界本原问题建立听众兴趣”“从水、火、存在等直观例子切入”等指导；为“苏格拉底与可对话的智能体”生成“Agent 协作解释”阶段，并提示用苏格拉底式对话类比多智能体协作。由此，编导计划成为脚本生成前的一层结构化约束。

#### 4.3 Agent Speechers: 角色化对谈生成

Agent Speechers 阶段根据编导计划生成双人对谈脚本。多智能体会话框架表明，不同智能体可以通过对话分工完成复杂任务 [3]。在教育播客场景中，双主持人并非简单轮流说话；他们分别承担“提问者”和“解释者”的职责：一方提出听众可能关心的问题，另一方负责解释概念、指出逻辑关系并抛出新的问题。

例如，在示例脚本中，A 先提出“古希腊人为什么总是在追问世界到底由什么构成”，B 则解释当水、火、存在等答案彼此冲突时，哲学问题会转向人如何认识、判断和共同生活。随后，脚本自然过渡到系统设计：一

个 Agent 负责拆解章节，另一个像编导一样安排讨论阶段，最后两个主持人 Agent 按计划展开对话。这个例子体现了对谈脚本将知识讲解和系统解释结合起来的方式。

#### 4.4 表现层：配音、画面与剪辑

表现层接收内容层产出的脚本，重点处理听感、画面和后期合成。它接收已经生成的对话内容，并将其加工成教育播客视频和配套文案。Voice 阶段先将 A/B 角色台词切分为 **AudioChunk**，每个 chunk 记录 **speaker**、**text**、**duration**、**start\_time** 和 **end\_time**。实际配音可接入 ChatTTS，也可接入本地 GPT-SoVITS 服务。后者在项目中承担 GPT voice 配音角色：系统根据说话人选择参考音频、提示文本和语言参数，并使用 **cut0** 模式，避免服务端重复切句。

长脚本配音的主要工程问题是等待时间和音色稳定性。项目采用 `chunk` 级并发机制：脚本先被拆成适合呼吸和停顿的短句，再由异步任务并发提交给 TTS 服务；GPT-SoVITS 请求通过 `semaphore` 控制为同时 2 路，以免本地显存和服务端压力过大。语义切句也可以交给 LLM 处理，并通过 Pydantic 结构化输出约束为 `speaker` 和 `text` 字段，减少后续配音阶段的解析歧义。

配音完成后，Export 节点采用“数学静音机制”修正听感和字幕时间轴。直接拼接所有音频数组会让句子之间过于紧凑，因此系统在相邻句子之间插入零数组静音；同一角色连续发言时停顿较短，角色切换时停顿较长。系统同步推进全局时间轴游标，再写出 MP3 和 SRT。这样生成的字幕时间码与真实音频长度一致，也能让对谈听起来更自然。

Image 阶段以 SRT 时间轴为输入,把字幕按时间和语义切分为视觉场景。系统可调用 Qwen/DashScope 文生图接口生成 1920×1080 场景图,也可在课程演示中使用静态图作为稳定背景。生成式配图带有限流退避和重试逻辑,静态图模式则更适合快速复现。

Editor 阶段负责把音频、字幕和图片合成为视频。代码中保留 MoviePy 与 FFmpeg 两种路径: MoviePy 适合多图片轨和字幕轨组合, FFmpeg 路径更轻量, 适合静态图、音频和 SRT 的快速合成, 并通过 `h264_nvenc`、AAC、恒定帧率和 `shortest` 等参数生成最终视频。由此, 系统最终输出教育播客视频和文案, 产物范围从文本扩展到可播放媒体。

#### 4.5 上下文组织机制

为了避免角色漂移和事实漂移，系统需要区分不同类型的上下文。System Prompt 用于固定角色、人设、表达风格和任务规则；Current Task 用于指定当前回合要回答的问题和要抛出的下一个问题；Memory 记录最近对话历史，使发言能够承接上文；Knowledge Base 则对应知识层，主要由参考章节和 RAG 检索结果构成，为知识性内容提供依据。

表 1: 上下文组织机制

| 上下文类型          | 生命周期  | 主要作用      | 风险控制      |
|----------------|-------|-----------|-----------|
| System Prompt  | 全局稳定  | 固定角色与任务边界 | 防止角色和风格漂移 |
| Current Task   | 单轮变化  | 约束当前回答目标  | 防止泛泛而谈    |
| Memory         | 短期滑动  | 保持对话连贯    | 避免前后断裂    |
| Knowledge Base | 按资料更新 | 提供事实依据    | 降低知识幻觉    |

在对话生成中，短期 Memory 更适合采用滑动窗口，而不必把每一句历史对话都放入向量检索。原因是对话连贯性依赖最近语境，过度检索可能带来语义断层。知识库则更适合用于章节资料和外部信息的约束；在本项目里，我把 RAG 放在知识层使用，没有把它当作所有上下文问题的统一解法。LangGraph 这类状态图

框架为长流程、持久化状态和人工介入提供了工程基础 [4], 因此适合承载这类多阶段 Agent 工作流。

除了自然语言上下文, 系统还需要格式上下文。Topic、Director、语义切句和场景切分等阶段都会要求模型输出 JSON 或结构化对象, 并在程序侧解析为 TopicItem、ExtendTopicItem、AudioChunk 等数据结构。这类格式约束把“模型说了什么”转化为“下游节点可以执行什么”, 也是多阶段工作流能稳定串联的基础。

## 5 原型实现与案例分析

本文使用一个哲学教育播客原型作为案例。输入材料面向西方哲学史讲解, 系统输出覆盖主题、编导、脚本和媒体阶段摘要。该案例不以大规模实验为目标, 而用于说明多阶段 Agent 工作流的产物形态和可检查性。

表 2: 阶段职责与产物

| 阶段              | 输入     | 输出      | 作用         |
|-----------------|--------|---------|------------|
| Topic           | 参考章节   | 主题列表    | 建立从浅入深的内容链 |
| Director        | 主题列表   | 编导计划    | 规定段落意图和转场  |
| Agent Speechers | 编导计划   | 双人脚本    | 生成可播出的对谈文本 |
| Voice           | 脚本     | 音频/字幕摘要 | 接入语音生成     |
| Image           | 脚本或时间轴 | 画面计划    | 组织视觉素材     |
| Editor          | 音频与画面  | 视频摘要或成片 | 完成媒体组合     |

在案例产物中, Topic 阶段输出了两个主题: “从自然追问到人的转向”和“苏格拉底与可对话的智能体”。前者强调从世界本原问题过渡到人的判断, 后者强调苏格拉底式问答与 Agent 协作之间的类比。Director 阶段进一步为主题生成“问题导入”和“Agent 协作解释”等讨论阶段, 明确每段意图和转场方式。

脚本阶段的示例片段如下:

- A: 我们先从一个很朴素的问题开始: 古希腊人为什么总是在追问世界到底由什么构成?  
 B: 当水、火、存在这些答案彼此冲突时, 哲学就会意识到, 单靠争论一个终极本原并不够。于是问题开始转向人如何认识、如何判断、如何共同生活。

这一片段先从听众可理解的问题出发, 再逐步引出抽象概念, 避免直接堆砌哲学术语。后续 Voice、Image 和 Editor 阶段则将脚本连接到配音、画面计划和视频组织, 为完整媒体生产保留接口。

## 6 工程化分析与讨论

多阶段 Agent 工作流的价值主要体现在过程可控。第一, 中间产物提高了可解释性。通过观察 Topic、Director 和脚本, 开发者或审核者可以知道系统如何从参考材料走向节目内容。第二, 阶段化设计提高了可调试性。当某一阶段输出质量不足时, 可以单独修改该阶段提示词或人工调整该阶段产物, 而不必重新设计整条链路。第三, 模块化结构提高了可扩展性。底层大模型、语音合成、图像生成和视频剪辑工具都可以被替换, 而不影响整体流程抽象。

从实现体验看, 这种拆分最大的好处是出错位置比较明确。此外, 该设计也为人机协同留下空间。对于知识型内容, 完全自动生成并不总是最优选择。更可行的方式是在 Topic 或 Director 阶段插入人工审核, 由人确认主题结构和编导方向, 再让 Agent 生成具体脚本。这种“机器生成结构草案、人类把控关键节点”的模式, 能够在效率与可靠性之间取得平衡。

系统也存在局限。首先, 真实全链路依赖大模型 API、语音合成、图像生成和视频处理工具, 外部服务稳定性会影响运行体验。其次, 内容质量仍依赖参考资料质量和提示词设计, 不能完全消除模型幻觉。最后, 当

前案例分析主要展示流程和产物形态, 尚未建立系统化评价指标。后续工作可引入人工评分、事实一致性检查和听众理解度评估, 以更全面地衡量生成质量。

## 7 AI 使用说明

本项目和本报告的整理过程中使用了 AI 辅助。具体包括三类: 第一, 在工程整理阶段辅助生成部分前端代码和界面交互代码; 第二, 辅助撰写和修改内容生成提示词, 使 Topic、Director 和 Agent Speechers 等阶段的任务描述更清晰; 第三, 辅助对报告文字进行润色和格式整理。

## 8 总结

本文围绕选题 E “工程实践与设计”, 分析了一个面向教育播客生成的多阶段 Agent workflows 原型。该系统将知识型播客生产拆解为主题规划、编导计划、角色化脚本生成和媒体产物组织等阶段, 避免将长文本生成压力集中在一次模型调用中。通过案例可以看到, 结构化中间产物使系统具备较好的可解释性、可检查性和扩展性。该实践说明, 大模型 Agent 在内容生产中的价值不仅体现在文本生成能力, 也体现在将复杂任务组织为可运行 workflows 的工程方法上。

## 参考文献

- [1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations. 2023.
- [2] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems. 2020.
- [3] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[EB/OL]. arXiv:2308.08155, 2023.
- [4] LangChain. LangGraph Documentation: Overview[EB/OL]. <https://docs.langchain.com/oss/python/langgraph/overview>.